

Version 2 block-download scheduler specification.

Models the layer above the per-connection protocol in *protocol.tla*: the node choosing which peer to ask for each block — the layer where Zebra’s genesis-to-tip sync stall lived in the legacy protocol (*ZcashFoundation/zebra* $\neq$ 10679; see *../sync_scheduler.tla*). Zebra’s *v2* stack defers this layer to a future scheduler (“ibd-engine”), so this module models it ahead of the implementation.

The *v2* protocol gives a request more ways to end without a block than the legacy protocol did. Only one of them says anything about the peer’s chain:

not-found	the per-entry result 0x02 of get-blocks: an explicit statement that the peer lacks the block.
REFUSED	the responder reset the stream, declining to serve it (overload, rate limits, Zebra’s bulk-stream cap). Says nothing about the peer’s chain.
truncation	the responder finished after any complete entry, leaving later entries unanswered (“get-blocks”, “get-block-range”). Routine and sanctioned; says nothing about the peer’s chain.
timeout	no response within the stall timeout; the requester cancels with <i>CANCELLED</i> (“Block Download Parameters”). Says nothing about the peer’s chain.

Three booleans mark each of the uninformative outcomes as informative, each reproducing a variant of the legacy registry-poisoning bug:

<i>TreatRefusedAsMissing</i>	= TRUE $\rightarrow$ a REFUSED marks the peer missing
<i>TreatTruncatedAsMissing</i>	= TRUE $\rightarrow$ an unanswered entry marks the peer missing
<i>BuggyTimeout</i>	= TRUE $\rightarrow$ a timeout marks the peer missing (the exact legacy zebra $\neq$ 10679 bug)

Separately, the scheduler applies Zebra’s unresponsive-peer rule: a peer that times out *UnresponsiveLimit* requests in a row, with no response of any kind in between, is disconnected (*ZcashFoundation/zebra* $\neq$ 11276). The *Redial* boolean decides whether disconnected peers are ever redialled; without it, evicting the only holder of a block stalls the sync even though every registry entry is honest.

RegistryHonest and *EventuallyAllVerified* hold with every switch in its fixed position (*Redial* = TRUE, the rest FALSE) and are violated under the buggy settings, each as a short *TLC* counterexample.

See *../documents/v2-modeling.md* for the full write-up.

EXTENDS *Naturals*, *FiniteSets*

CONSTANTS

set of peer ids
@type: *Set(Str)*;

Peers,

blocks to download are 1 .. *MaxBlock*
@type: *Int*;

MaxBlock,

peers that hold only blocks 1 .. *LagTip*
@type: *Set(Str)*;

LaggingPeers,

chain height of a lagging peer (< *MaxBlock*)
@type: *Int*;

LagTip,
 bound on re-queue attempts per block after a notfound
 @type: *Int*;
MaxRetries,
 consecutive timeouts before a peer is disconnected
 @type: *Int*;
UnresponsiveLimit,
 TRUE treats a REFUSED as a notfound (buggy)
 @type: *Bool*;
TreatRefusedAsMissing,
 TRUE treats an unanswered entry as a notfound (buggy)
 @type: *Bool*;
TreatTruncatedAsMissing,
 TRUE treats a timeout as a notfound (legacy bug)
 @type: *Bool*;
BuggyTimeout,
 TRUE redials disconnected peers
 @type: *Bool*;
Redial

Blocks $\triangleq 1 \dots \text{MaxBlock}$
NONE $\triangleq$ “none” sentinel for “no peer”; must not be a member of *Peers*

Ground truth: a lagging peer holds blocks up to *LagTip*; every other peer is at tip and holds them all. The scheduler never reads this directly — it learns a peer’s contents only through the outcomes below.

PeerTip(*p*) $\triangleq$ IF *p* $\in$ *LaggingPeers* THEN *LagTip* ELSE *MaxBlock*
Holds(*p*, *b*) $\triangleq b \leq \text{PeerTip}(p)$

VARIABLES

@type: *Set(Int)*;
verified, subset of *Blocks* downloaded and verified (the frontier)
 @type: *Set(Int)*;
pending, subset of *Blocks* the scheduler still wants and may request
 @type: *Int* $\rightarrow$ *Str*;
inflight, [*Blocks* $\rightarrow$ *Peers* $\cup$ {*NONE*}] peer a block is requested from
 @type: *Str* $\rightarrow$ (*Int* $\rightarrow$ *Str*);
avail, [*Peers* $\rightarrow$ [*Blocks* $\rightarrow$ {“has”, “missing”, “unknown”}]] registry
 @type: *Int* $\rightarrow$ *Int*;
retries, [*Blocks* $\rightarrow 0 \dots \text{MaxRetries}$] re-queue attempts used
 @type: *Str* $\rightarrow$ *Int*;
timeouts, [*Peers* $\rightarrow 0 \dots \text{UnresponsiveLimit}$] consecutive unanswered timeouts
 @type: *Str* $\rightarrow$ *Bool*;
connected [*Peers* $\rightarrow$ BOOLEAN] FALSE once disconnected as unresponsive

vars $\triangleq \langle \text{verified}, \text{pending}, \text{inflight}, \text{avail}, \text{retries}, \text{timeouts}, \text{connected} \rangle$

Init $\triangleq$

$\wedge \text{verified} = \{\}$
 $\wedge \text{pending} = \text{Blocks}$
 $\wedge \text{inflight} = [b \in \text{Blocks} \mapsto \text{NONE}]$
 $\wedge \text{avail} = [p \in \text{Peers} \mapsto [b \in \text{Blocks} \mapsto \text{"unknown"}]]$
 $\wedge \text{retries} = [b \in \text{Blocks} \mapsto 0]$
 $\wedge \text{timeouts} = [p \in \text{Peers} \mapsto 0]$
 $\wedge \text{connected} = [p \in \text{Peers} \mapsto \text{TRUE}]$

The scheduler opens a get-blocks request for a pending block toward a connected peer not already marked missing for it.

Request $\triangleq$

$\exists b \in \text{pending} :$
 $\exists p \in \text{Peers} :$
 $\wedge \text{inflight}[b] = \text{NONE}$
 $\wedge \text{connected}[p]$
 $\wedge \text{avail}[p][b] \neq \text{"missing"}$
 $\wedge \text{inflight}' = [\text{inflight} \text{ EXCEPT } ![b] = p]$
 $\wedge \text{pending}' = \text{pending} \setminus \{b\}$
 $\wedge \text{UNCHANGED } \langle \text{verified}, \text{avail}, \text{retries}, \text{timeouts}, \text{connected} \rangle$

The in-flight peer genuinely has the block: it is delivered and verified.

Any response resets the peer's consecutive-timeout count.

DeliverBlock $\triangleq$

$\exists b \in \text{Blocks} :$
 $\text{LET } p \triangleq \text{inflight}[b] \text{ IN}$
 $\wedge p \neq \text{NONE}$
 $\wedge \text{Holds}(p, b)$
 $\wedge \text{verified}' = \text{verified} \cup \{b\}$
 $\wedge \text{avail}' = [\text{avail} \text{ EXCEPT } ![p][b] = \text{"has"}]$
 $\wedge \text{inflight}' = [\text{inflight} \text{ EXCEPT } ![b] = \text{NONE}]$
 $\wedge \text{timeouts}' = [\text{timeouts} \text{ EXCEPT } ![p] = 0]$
 $\wedge \text{UNCHANGED } \langle \text{pending}, \text{retries}, \text{connected} \rangle$

The in-flight peer genuinely lacks the block and answers the entry with the explicit not-found result. This is the one outcome that legitimately marks the registry, and the block is re-queued (bounded) toward other peers.

NotFound $\triangleq$

$\exists b \in \text{Blocks} :$
 $\text{LET } p \triangleq \text{inflight}[b] \text{ IN}$
 $\wedge p \neq \text{NONE}$
 $\wedge \neg \text{Holds}(p, b)$
 $\wedge \text{inflight}' = [\text{inflight} \text{ EXCEPT } ![b] = \text{NONE}]$
 $\wedge \text{avail}' = [\text{avail} \text{ EXCEPT } ![p][b] = \text{"missing"}]$

$\wedge \text{timeouts}' = [\text{timeouts} \text{ EXCEPT } ![p] = 0]$
 $\wedge \text{IF } \text{retries}[b] < \text{MaxRetries}$
 THEN $\wedge \text{pending}' = \text{pending} \cup \{b\}$
 $\wedge \text{retries}' = [\text{retries} \text{ EXCEPT } ![b] = @ + 1]$
 ELSE $\wedge \text{UNCHANGED } \text{pending}$ retries exhausted
 $\wedge \text{UNCHANGED } \text{retries}$
 $\wedge \text{UNCHANGED } \langle \text{verified}, \text{connected} \rangle$

The responder resets the request stream with REFUSED — it declines to serve the request, whether or not it holds the block (draft: “Request Streams”;
Zebra refuses beyond 2 concurrent bulk streams). A REFUSED is a response,
so the timeout count resets; what it does to the registry is the
TreatRefusedAsMissing switch.

$\text{Refused} \triangleq$
 $\exists b \in \text{Blocks} :$
 LET $p \triangleq \text{inflight}[b]$ IN
 $\wedge p \neq \text{NONE}$
 $\wedge \text{inflight}' = [\text{inflight} \text{ EXCEPT } ![b] = \text{NONE}]$
 $\wedge \text{pending}' = \text{pending} \cup \{b\}$
 $\wedge \text{timeouts}' = [\text{timeouts} \text{ EXCEPT } ![p] = 0]$
 $\wedge \text{avail}' = [\text{avail} \text{ EXCEPT } ![p][b] =$
 IF *TreatRefusedAsMissing* THEN “missing” ELSE @]
 $\wedge \text{UNCHANGED } \langle \text{verified}, \text{retries}, \text{connected} \rangle$

The responder finishes the stream early, leaving this entry unanswered
(draft: “get-blocks” / “get-block-range” allow finishing after any complete
entry; truncation is resumable). Sanctioned and routine — but uninformative
about the peer’s chain. The registry effect is the *TreatTruncatedAsMissing*
switch.

$\text{Truncated} \triangleq$
 $\exists b \in \text{Blocks} :$
 LET $p \triangleq \text{inflight}[b]$ IN
 $\wedge p \neq \text{NONE}$
 $\wedge \text{inflight}' = [\text{inflight} \text{ EXCEPT } ![b] = \text{NONE}]$
 $\wedge \text{pending}' = \text{pending} \cup \{b\}$
 $\wedge \text{timeouts}' = [\text{timeouts} \text{ EXCEPT } ![p] = 0]$
 $\wedge \text{avail}' = [\text{avail} \text{ EXCEPT } ![p][b] =$
 IF *TreatTruncatedAsMissing* THEN “missing” ELSE @]
 $\wedge \text{UNCHANGED } \langle \text{verified}, \text{retries}, \text{connected} \rangle$

No response within the stall timeout: the requester cancels the stream with
CANCELLED and re-queues the block (draft: “Block Download Parameters”).
The registry effect is the *BuggyTimeout* switch — the exact legacy bug.
The peer’s consecutive-timeout count rises; at *UnresponsiveLimit* the peer
is disconnected as unresponsive (Zebra’s rule).

$\text{Timeout} \triangleq$

$$\begin{aligned}
& \exists b \in \text{Blocks} : \\
& \quad \text{LET } p \triangleq \text{inflight}[b] \text{ IN} \\
& \quad \wedge p \neq \text{NONE} \\
& \quad \wedge \text{inflight}' = [\text{inflight} \text{ EXCEPT } ![b] = \text{NONE}] \\
& \quad \wedge \text{pending}' = \text{pending} \cup \{b\} \\
& \quad \wedge \text{avail}' = [\text{avail} \text{ EXCEPT } ![p][b] = \\
& \quad \quad \text{IF } \text{BuggyTimeout} \text{ THEN "missing" ELSE } @] \\
& \quad \wedge \text{timeouts}' = [\text{timeouts} \text{ EXCEPT } ![p] = \text{IF } @ < \text{UnresponsiveLimit} \text{ THEN } @ + 1 \text{ ELSE } @] \\
& \quad \wedge \text{connected}' = [\text{connected} \text{ EXCEPT } ![p] = @ \wedge \text{timeouts}[p] + 1 < \text{UnresponsiveLimit}] \\
& \quad \wedge \text{UNCHANGED } \langle \text{verified}, \text{retries} \rangle
\end{aligned}$$

A disconnected peer is redialled (Zebra maintains connections to its initial and discovered peers, redialling them when they fail). Gated by the *Redial* switch; without it eviction is forever.

$$\begin{aligned}
\text{Reconnect} & \triangleq \\
& \exists p \in \text{Peers} : \\
& \quad \wedge \text{Redial} \\
& \quad \wedge \neg \text{connected}[p] \\
& \quad \wedge \text{connected}' = [\text{connected} \text{ EXCEPT } ![p] = \text{TRUE}] \\
& \quad \wedge \text{timeouts}' = [\text{timeouts} \text{ EXCEPT } ![p] = 0] \\
& \quad \wedge \text{UNCHANGED } \langle \text{verified}, \text{pending}, \text{inflight}, \text{avail}, \text{retries} \rangle
\end{aligned}$$

$$\begin{aligned}
\text{Next} & \triangleq \\
& \vee \text{Request} \\
& \vee \text{DeliverBlock} \\
& \vee \text{NotFound} \\
& \vee \text{Refused} \\
& \vee \text{Truncated} \\
& \vee \text{Timeout} \\
& \vee \text{Reconnect}
\end{aligned}$$

Strong fairness on the progress actions encodes the real-world assumption that the node keeps trying, a willing peer eventually responds, and the redial loop keeps running. Refused, *Truncated* and *Timeout* get no fairness — they are adversarial but never forced.

$$\begin{aligned}
\text{Spec} & \triangleq \\
& \text{Init} \\
& \wedge \Box [\text{Next}]_{\text{vars}} \\
& \wedge \text{SF}_{\text{vars}}(\text{Request}) \\
& \wedge \text{SF}_{\text{vars}}(\text{DeliverBlock}) \\
& \wedge \text{SF}_{\text{vars}}(\text{NotFound}) \\
& \wedge \text{SF}_{\text{vars}}(\text{Reconnect})
\end{aligned}$$

Liveness: every block is eventually downloaded and verified.
 $EventuallyAllVerified \triangleq \Diamond(\text{verified} = \text{Blocks})$

Safety invariants.

$TypeOK \triangleq$

- $\wedge \text{verified} \subseteq \text{Blocks}$
- $\wedge \text{pending} \subseteq \text{Blocks}$
- $\wedge \text{inflight} \in [\text{Blocks} \rightarrow \text{Peers} \cup \{NONE\}]$
- $\wedge \text{avail} \in [\text{Peers} \rightarrow [\text{Blocks} \rightarrow \{\text{"has"}, \text{"missing"}, \text{"unknown"}\}]]$
- $\wedge \text{retries} \in [\text{Blocks} \rightarrow 0 \dots \text{MaxRetries}]$
- $\wedge \text{timeouts} \in [\text{Peers} \rightarrow 0 \dots \text{UnresponsiveLimit}]$
- $\wedge \text{connected} \in [\text{Peers} \rightarrow \text{BOOLEAN}]$

The registry never marks a peer missing for a block it actually holds:
 only the explicit not-found result may mark it. “A refusal, a truncated
 response, and a timeout are not notfound.”

$RegistryHonest \triangleq$

- $\forall p \in \text{Peers} :$
- $\forall b \in \text{Blocks} :$
- $\text{avail}[p][b] = \text{"missing"} \Rightarrow \neg \text{Holds}(p, b)$

A peer is only ever recorded as having a block it genuinely holds.

$RegistryHasIsSound \triangleq$

- $\forall p \in \text{Peers} :$
- $\forall b \in \text{Blocks} :$
- $\text{avail}[p][b] = \text{"has"} \Rightarrow \text{Holds}(p, b)$

A verified block was genuinely available from some peer.

$VerifiedAreReal \triangleq$

- $\forall b \in \text{verified} :$
- $\exists p \in \text{Peers} : \text{Holds}(p, b)$
